

Projet DATAImmo – Analyse des transactions immobilières

Roger AGNEROH

Formation Data Analyst – OpenClassrooms

Laplace Immo

1-Contexte du projet

- Analyse du marché immobilier français
- Données DVF + données INSEE
- Objectif : aide à la décision



2-Objectifs

- Construire une base fiable
- Analyser le marché immobilier
- Produire des indicateurs métier

3- La stratégie de sauvegarde et la conformité RGPD

- Données anonymisées (DVF)
- Aucune donnée personnelle
- Utilisation open data
- Conforme RGPD

4- Méthodologie

- Modélisation (3NF)
- Nettoyage des données
- Intégration SQL
- Analyse via requêtes

5- Modèle de données

- Tables principales :
  - vente
  - bien
  - commune
  - Region

- Relations via clés primaires / étrangères

6- Données

- DVF : transactions immobilières
- INSEE : population
- Données nettoyées et intégrées

6-L'extrait du dictionnaire des données

DICTIONNAIRE DES DONNÉES - Données communes(INSEE)

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
id_codedep_codecommune	Identifiant unique de la commune (code INSEE conca	VARCHAR	10	Élémentaire	PK, NOT NULL, UNIQUE	
code_departement	Code du département (ex: 75, 13, 2A)	VARCHAR	3	Élémentaire	NOT NULL	
code_commune	Code officiel de la commune (INSEE)	INTEGER	NC	Élémentaire	NOT NULL	
code_postal	Code postal (peut être multiple selon les communes)	INTEGER	NC	Élémentaire	NULL autorisé	
nom_commune	Nom officiel de la commune	VARCHAR	50	Élémentaire	NOT NULL	
reg_code	Code de la région	VARCHAR	5	Élémentaire	FK → region.reg_code, NULL autorisé (si non renseigné dans la source)	
id_population	Identifiant de la ligne population	INTEGER	NC	Élémentaire	PK, NOT NULL, UNIQUE	
id_codedep_codecommune (pc	Commune associée aux indicateurs de population	VARCHAR	10	Élémentaire	FK → commune.id_codedep_codecommune, NOT NULL	
pmun	Population municipale (INSEE)	INTEGER	NC	Élémentaire	>= 0, NOT NULL	
pcap	Population comptée à part (INSEE)	INTEGER	NC	Élémentaire	>= 0, NOT NULL	
ptot	Population totale (INSEE)	INTEGER	NC	Calculée	>= 0, NOT NULL, cohérent avec PMUN + PCAP : PTOT = PMUN + PCAP	

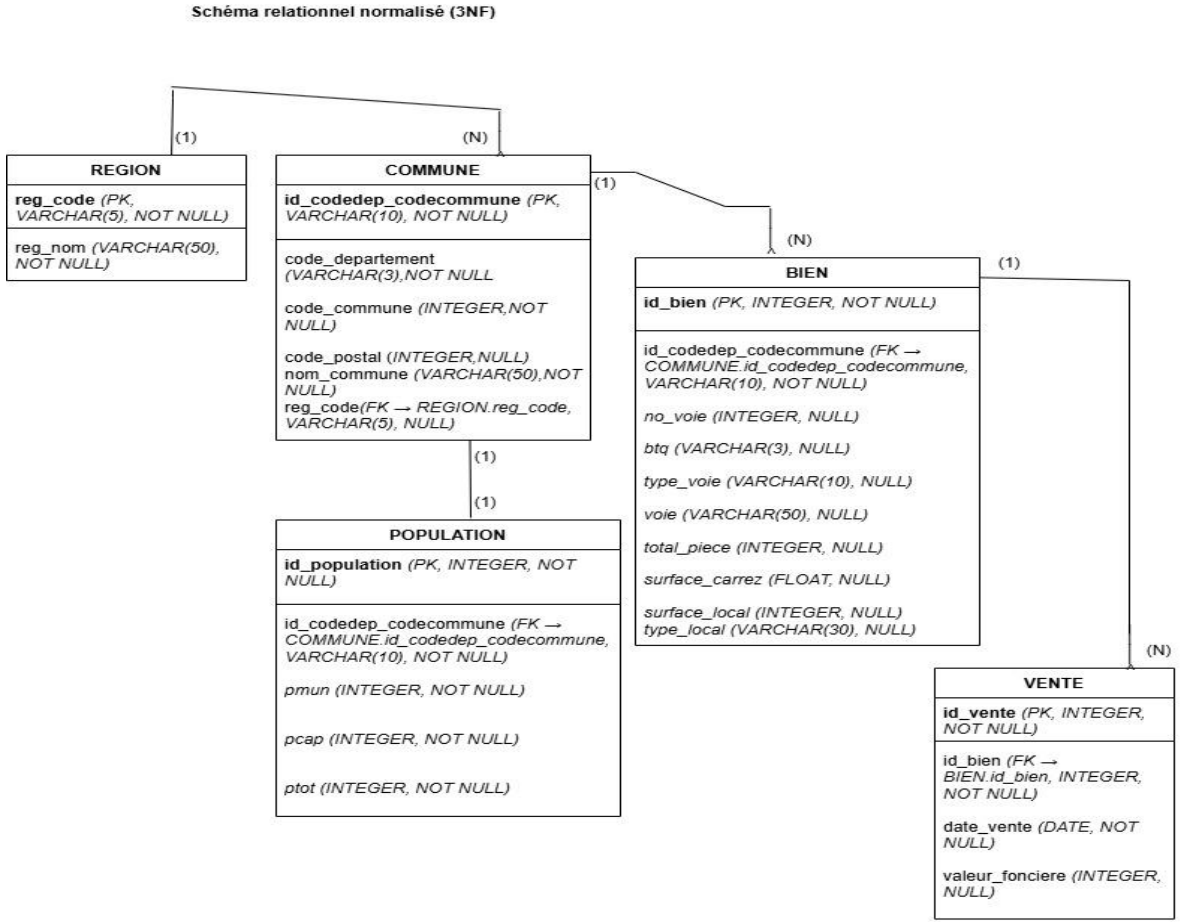
DICTIONNAIRE DES DONNÉES - Valeurs foncières

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
id_bien	Identifiant unique du bien immobilier	INTEGER	NC	Élémentaire	PK, NOT NULL, UNIQUE	
id_codedep_codecomr	Identifiant de la commune du bien (code INSEE concaténé dépa	VARCHAR	10	Élémentaire	FK → commune.id_codedep_codecommune, NOT NULL	
no_voie	Numéro dans la voie	INTEGER	NC	Élémentaire	>= 0, NULL autorisé	
btq	Indice de répétition (bis/ter/quater...)	VARCHAR	3	Élémentaire	NULL autorisé	
type_voie	Type de voie (RUE, AV, BD...)	VARCHAR	10	Élémentaire	NULL autorisé	
voie	Nom de la voie	VARCHAR	50	Élémentaire	NULL autorisé	
total_piece	Nombre de pièces principales	INTEGER	NC	Élémentaire	>= 0, NULL autorisé	
surface_carrez	Surface loi Carrez (m²)	FLOAT	NC	Élémentaire	>= 0, NULL autorisé	
surface_local	Surface du local (m²)	INTEGER	NC	Élémentaire	>= 0, NULL autorisé	
type_local	Type de local (Appartement, Maison...)	VARCHAR	30	Élémentaire	Valeurs attendues : 'Appartement', 'Maison', autres selon DVF	
id_vente	Identifiant unique de la vente	INTEGER	NC	Élémentaire	PK, NOT NULL, UNIQUE	
id_bien_vente	Identifiant du bien vendu	INTEGER	NC	Élémentaire	FK → bien.id_bien, NOT NULL	
date_vente	Date de mutation (vente)	DATE	NC	Élémentaire	NOT NULL	
valeur_fonciere	Valeur foncière / prix de vente (€)	INTEGER	NC	Élémentaire	>= 0, NULL autorisé	

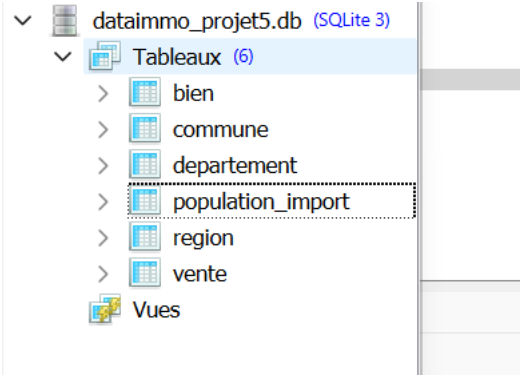
DICTIONNAIRE DES DONNÉES - Référentiel géographique

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
reg_code	Code officiel de la région	VARCHAR	5	Élémentaire	PK, NOT NULL, UNIQUE	
reg_nom	Nom officiel de la région	VARCHAR	50	Élémentaire	NOT NULL, libellé officiel INSEE	

7-Le schéma relationnel normalisé



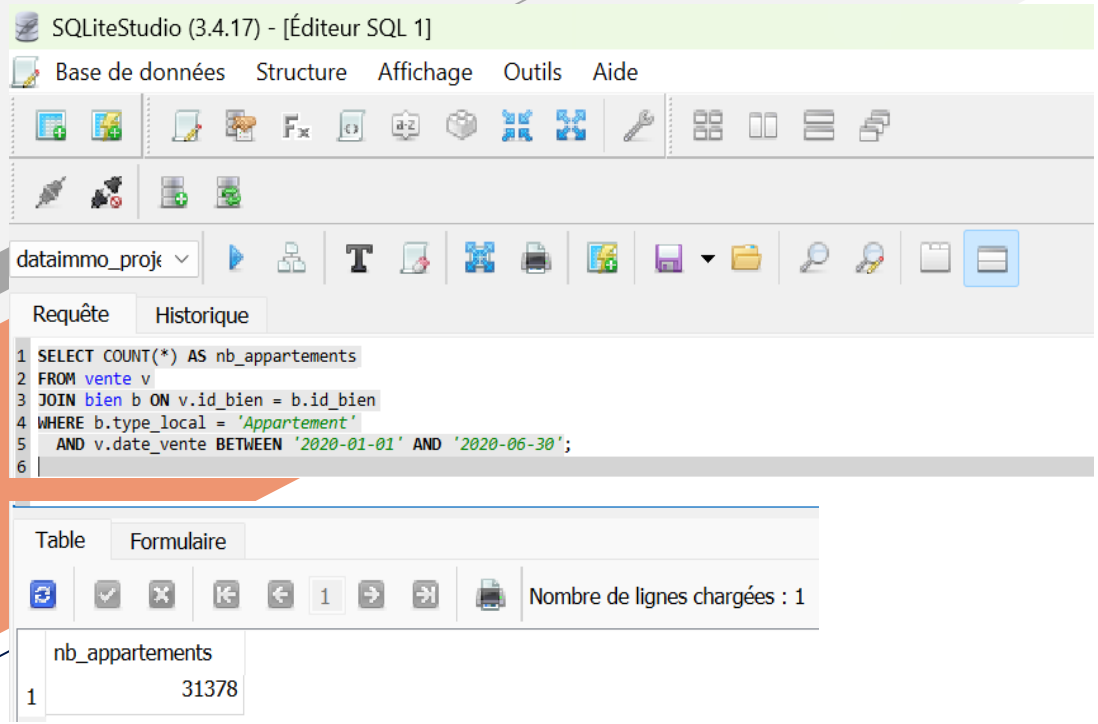
8-La base de données avec les tables créées et les données chargées



- Tables créées
- Données importées
- Jointures vérifiées

9- Exemples des 12 requêtes SQL sur les données

ANALYSE 1. Nombre total d'appartements vendus au 1er semestre 2020



JOIN vente ↔ bien
Filtre : appartement
Période : S1 2020
COUNT(*)

31 378 ventes : activité immobilière élevée

ANALYSE 2. Le nombre de ventes d'appartement par région pour le 1er semestre 2020

dataimmo_proje

Requête | Historique

```sql
SELECT
    c.reg_code,
    r.reg_nom,
    COUNT(*) AS nb_ventes
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
JOIN region r ON c.reg_code = r.reg_code
WHERE b.type_local = 'Appartement'
  AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
GROUP BY c.reg_code, r.reg_nom
ORDER BY nb_ventes DESC;
```

Table | Formulaire

1 | Nombre de lignes chargées : 12

	reg_code	reg_nom	nb_ventes
1	11	Ile-de-France	13995
2	84	Auvergne-Rhône-Alpes	2977
3	93	Provence-Alpes-Côte d'Azur	2153
4	75	Nouvelle-Aquitaine	1932
5	76	Occitanie	1628
6	52	Pays de la Loire	1357
7	32	Hauts-de-France	1183
8	53	Bretagne	983

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune ↔ region)
- filtré sur type_local = 'Appartement'
- bien limité sur la période demandée
- regroupé par région
- utilisé COUNT(*)

13 995 ventes en **Île-de-France** : forte concentration du marché

ANALYSE 3. Proportion des ventes d'appartements par le nombre de pièces.

dataimmo_proje

Requête | Historique

```sql
SELECT
    b.nombre_pieces_principales,
    COUNT(*) AS nb_ventes,
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*)
         FROM vente v
         JOIN bien b ON v.id_bien = b.id_bien
         WHERE b.type_local = 'Appartement'
            ),2
    ) AS proportion_pct
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
WHERE b.type_local = 'Appartement'
 GROUP BY b.nombre_pieces_principales
ORDER BY b.nombre_pieces_principales;
```

Table | Formulaire

Nombre de lignes chargées : 12

	nombre_pieces_principales	nb_ventes	proportion_pct
1	0	30	0.1
2	1	6739	21.48
3	2	9783	31.18
4	3	8966	28.57
5	4	4460	14.21
6	5	1114	3.55
7	6	204	0.65
8	7	54	0.17

J'ai :

- utilisé un JOIN (vente ↔ bien)
- filtré sur type_local = 'Appartement'
- bien limité sur la période demandée
- regroupé par nombre de pièces
- calculé un pourcentage

2 pièces (31%) et 3 pièces (28%) : forte demande sur petites surfaces.

ANALYSE 4. Liste des 10 départements où le prix du mètre carré est le plus élevé.

```sql
SELECT
    c.code_departement,
    ROUND(AVG(v.valeur_fonciere * 1.0 / b.surface_reelle_bati), 2) AS prix_m2
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
WHERE b.surface_reelle_bati > 0
  AND v.valeur_fonciere IS NOT NULL
  AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
GROUP BY c.code_departement
ORDER BY prix_m2 DESC
LIMIT 10;
```

Table | Formulaire

Nombre de lignes chargées : 10

	code_departement	prix_m2
1	75	12121.88
2	92	7415.28
3	94	5395.98
4	93	4363.01
5	74	4149.56
6	78	4126.19
7	69	4063.83
8	33	3838.74

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune)
- calculé prix m² = valeur_fonciere / surface
- exclu valeurs nulles
- regroupé par département
- trié DESC + LIMIT 10

Paris (75) : 12 121 €/m² → marché le plus cher.

ANALYSE 5. Prix moyen du mètre carré d'une maison en Île-de-France

dataimmo_proje

Requête | Historique

```
1  SELECT
2  ROUND(AVG(1.0 * v.valeur_fonciere / b.surface_reelle_bati),2) AS prix_m2_maison
3  FROM vente v
4  JOIN bien b ON v.id_bien = b.id_bien
5  JOIN commune c ON b.code_commune = c.code_commune
6  WHERE b.type_local = 'Maison'
7  AND b.surface_reelle_bati > 0
8  AND v.valeur_fonciere IS NOT NULL
9  AND c.reg_code = 11
10  AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30';
```

Table | Formulaire

1 | Nombre de lignes chargées : 1

	prix_m2_maison
1	3997.71

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune)
- filtré type_local = 'Maison'
- filtré sur région Île-de-France
- exclu valeurs nulles
- utilisé AVG

≈ 4 000 €/m² : marché maison très valorisé

ANALYSE 6. Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés

dataimmo_proje

Requête | Historique

```sql
SELECT
    v.valeur_fonciere,
    b.surface_reelle_bati,
    c.nom_commune,
    r.reg_nom
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
JOIN region r ON c.reg_code = r.reg_code
WHERE b.type_local = 'Appartement'
  AND v.valeur_fonciere IS NOT NULL
  AND b.surface_reelle_bati > 0
  AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
ORDER BY CAST(v.valeur_fonciere AS REAL) DESC
LIMIT 10;
```

Table | Formulaire

Nombre de lignes chargées : 10

	valeur_fonciere	surface_reelle_bati	nom_commune	reg_nom
1	9000000	10	Paris 16e	Ile-de-France
2	8600000	62	Corbeil-Essonnes	Ile-de-France
3	8577713	289	Paris 7e	Ile-de-France
4	7620000	42	Paris 17e	Ile-de-France
5	7600000	200	Paris 6e	Ile-de-France
6	7535000	143	Paris 1er	Ile-de-France
7	7420000	357	Paris 16e	Ile-de-France

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune ↔ region)
- filtré type_local = 'Appartement'
- exclu valeurs nulles
- trié par valeur_fonciere DESC
- utilisé LIMIT 10

Jusqu'à 9M€ : biens de luxe concentrés à Paris

ANALYSE 7. Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020

dataimmo_proje

Requête | Historique

```sql
WITH ventes AS (
    SELECT
        CASE
            WHEN strftime('%m', v.date_vente) IN ('01','02','03') THEN 'T1'
            WHEN strftime('%m', v.date_vente) IN ('04','05','06') THEN 'T2'
        END AS trimestre
    FROM vente v
    WHERE v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
)
SELECT ROUND(
    100.0 * (
        SUM(CASE WHEN trimestre = 'T2' THEN 1 ELSE 0 END) -
        SUM(CASE WHEN trimestre = 'T1' THEN 1 ELSE 0 END)
    ) / SUM(CASE WHEN trimestre = 'T1' THEN 1 ELSE 0 END),
    2
) AS evolution_pct
FROM ventes;
```

Table | Formulaire

1 | Nombre de lignes chargées : 1

	evolution_pct
1	3.68

J'ai :

- utilisé un découpage par trimestre
- compté les ventes T1 et T2
- calculé un taux d'évolution
- utilisé SUM + CASE

+3,68% : légère reprise de l'activité

ANALYSE 8. Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces

dataimmo_proj

Requête | Historique

```sql
SELECT
    c.reg_code,
    r.reg_nom,
    ROUND(AVG(CAST(v.valeur_fonciere AS REAL) * 1.0 / b.surface_reelle_bati), 2) AS prix_m2
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
JOIN region r ON c.reg_code = r.reg_code
WHERE b.type_local = 'Appartement'
  AND b.nombre_pieces_principales > 4
  AND b.surface_reelle_bati > 0
  AND v.valeur_fonciere IS NOT NULL
  AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
GROUP BY c.reg_code, r.reg_nom
ORDER BY prix_m2 DESC;
```

Table | Formulaire

1 | Nombre de lignes chargées : 12

	reg_code	reg_nom	prix_m2
1	11	Ile-de-France	8003.39
2	84	Auvergne-Rhône-Alpes	2899.24
3	75	Nouvelle-Aquitaine	2510.18
4	32	Hauts-de-France	2412.29
5	53	Bretagne	2271.86
6	93	Provence-Alpes-Côte d'Azur	2251.92

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune ↔ region)
- filtré appartements > 4 pièces
- exclu valeurs nulles
- calculé AVG prix m²
- regroupé par région

Île-de-France : région la plus chère (~8000 €/m²)

ANALYSE 9. Liste des communes ayant eu au moins 50 ventes au 1er trimestre

dataimmo_proje

Requête | Historique

```sql
SELECT
    c.nom_commune,
    c.code_commune,
    COUNT(*) AS nb_ventes
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
WHERE b.type_local = 'Appartement'
  AND v.date_vente BETWEEN '2020-01-01' AND '2020-03-31'
GROUP BY c.nom_commune, c.code_commune
HAVING COUNT(*) >= 50
ORDER BY nb_ventes DESC;
```

Table | Formulaire

1 | Nombre de lignes chargées : 44

	nom_commune	code_commune	nb_ventes
1	Paris 17e	75117	228
2	Paris 15e	75115	215
3	Paris 18e	75118	209
4	Paris 11e	75111	169
5	Paris 16e	75116	165
6	Bordeaux	33063	152
7	Paris 14e	75114	146
8	Paris 20e	75120	127

J'ai :

- utilisé des JOIN (vente ↔ bien ↔ commune)
- filtré appartements
- bien limité sur la période
- regroupé par commune
- utilisé HAVING COUNT >= 50

Paris domine : forte concentration des ventes

ANALYSE 10. Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces

dataimmo_proje

Requête | Historique

```sql
WITH prix_m2 AS (
    SELECT
        b.nombre_pieces_principales,
        AVG(CAST(v.valeur_fonciere AS REAL) * 1.0 / b.surface_reelle_bati) AS prix_m2
    FROM vente v
    JOIN bien b ON v.id_bien = b.id_bien
    WHERE b.type_local = 'Appartement'
      AND b.surface_reelle_bati > 0
      AND v.valeur_fonciere IS NOT NULL
      AND b.nombre_pieces_principales IN (2, 3)
      AND v.date_vente BETWEEN '2020-01-01' AND '2020-06-30'
    GROUP BY b.nombre_pieces_principales
)
SELECT ROUND(
    100.0 * (
        (SELECT prix_m2 FROM prix_m2 WHERE nombre_pieces_principales = 3) -
        (SELECT prix_m2 FROM prix_m2 WHERE nombre_pieces_principales = 2)
    ) /
    (SELECT prix_m2 FROM prix_m2 WHERE nombre_pieces_principales = 2),
    2
) AS difference_pct;
```

Table | Formulaire

1 | Nombre de lignes chargées : 1

	difference_pct
1	-13.04

J'ai :

- calculé prix m² moyen par type
- filtré 2 et 3 pièces
- utilisé une requête intermédiaire
- calculé différence en %

-13% : les petites surfaces sont plus chères

ANALYSE 11. Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69

dataimmo_proje

Requête | Historique

```sql
WITH moyenne_communes AS (
    SELECT
        c.nom_commune,
        c.code_commune,
        c.code_departement,
        ROUND(AVG(CAST(v.valeur_fonciere AS REAL)), 2) AS valeur_moyenne
    FROM vente v
    JOIN bien b ON v.id_bien = b.id_bien
    JOIN commune c ON b.code_commune = c.code_commune
    WHERE c.code_departement IN ('06', '13', '33', '59', '69')
      AND v.valeur_fonciere IS NOT NULL
    GROUP BY c.nom_commune, c.code_commune, c.code_departement
),
classement AS (
    SELECT
        nom_commune,
        code_commune,
        code_departement,
        valeur_moyenne,
        ROW_NUMBER() OVER (
            PARTITION BY code_departement
            ORDER BY valeur_moyenne DESC
        ) AS rang
    FROM moyenne_communes
)
SELECT
    nom_commune,
    code_commune,
    code_departement,
    valeur_moyenne,
    rang
FROM classement
WHERE rang <= 3
ORDER BY code_departement, rang;
```

Table | Formulaire

Nombre de lignes chargées : 12

	nom_commune	code_commune	code_departement	valeur_moyenne	rang
1	Gignac-la-Nerthe	13043	13	330000	1
2	Saint-Savournin	13101	13	314425	2
3	Cassis	13022	13	313416.88	3
4	Lège-Cap-Ferret	33236	33	549500.64	1
5	Vayres	33539	33	335000	2
6	Arcachon	33009	33	307435.93	3

JOIN tables
Filtre départements
AVG valeur foncière
ROW_NUMBER()
Top 3

Communes premium : zones les plus attractives

ANALYSE 12. Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants

dataimmo_proj

Requête | Historique

```
SELECT
    c.nom_commune,
    COUNT(*) * 1000.0 / c.population AS transactions_pour_1000
FROM vente v
JOIN bien b ON v.id_bien = b.id_bien
JOIN commune c ON b.code_commune = c.code_commune
WHERE c.population > 10000
GROUP BY c.nom_commune, c.population
ORDER BY transactions_pour_1000 DESC
LIMIT 20;
```

Table | Formulaire

Nombre de lignes chargées : 20

	nom_commune	transactions_pour_1000
1	Paris 2e	5.84311019093628
2	Paris 1er	4.92058548738711
3	Paris 3e	4.6930566081735
4	Arcachon	4.62262565136998
5	La Baule-Escoublac	4.5841519318926
6	Paris 4e	4.08302143586254
7	Paris 8e	3.83448275862069
8	Sanary-sur-Mer	3.4965034965035

J'ai :

- intégré la population dans commune
- utilisé des JOIN (vente ↔ bien ↔ commune)
- filtré population > 10 000
- calculé COUNT * 1000 / population
- trié DESC

Paris et zones touristiques : marché le plus dynamique



10-Conclusion

- Base de données fiable et normalisée
- Analyses SQL pertinentes
- Insights métier exploitables
- Compétences Data Analyst développées